JOURNAL OF COMPUTER AND SYSTEM SCIENCES 34, 19–26 (1987)

Upper Bounds for Time-Space Trade-offs in Sorting and Selection*

GREG N. FREDERICKSON

Department of Computer Sciences, Purdue University, West Lafayette, Indiana 47907

Received January 13, 1983; accepted July 7, 1986

Upper bound time-space trade-offs are established for sorting and selection in two computational models. For machines with input in read-only random access registers, and for machines with input on a read-only tape, we present algorithms that realize tradeoffs of $T \cdot S = O(N^2)$ for sorting and $T \log S = O(N \log N)$ for selection, where S is the number of workspace registers. © 1987 Academic Press, Inc.

1. INTRODUCTION

Recently, considerable work has been directed towards identifying the time-space trade-offs for a number of problems on various models of computation. In particular, the fundamental problems of sorting N elements or selecting an element of rank k from a set of N elements have been studied in [2–7] and [4], respectively. Trade-offs for sorting in an oblivious model of computation have been established in [5, 6]. Trade-offs for both sorting and selection in a tape-input model have been treated in [4]. Trade–offs for sorting in less restrictive models have been studied in [2, 3, 7]. With respect to the non-oblivious models, the upper bounds in the preceding papers are not entirely satisfactory. In this paper we present algorithms for sorting and selection that improve on previous upper bounds, in two models. Interestingly, the work highlights the differences in the nature of the trade-offs in sorting and selection, while uncovering no significant differences in the trade-offs due to the models themselves.

Both models are based on unit-cost RAM's. In the *register input model*, input values are stored in read-only random access registers, and output is to write-only random access registers. Workspace registers may hold addresses of input and output registers, and counts, and contain $\log N$ bits each.[1] To be consistent with the lower bound results that have been derived [1, 2], the space S_b will be counted as the number of bits in workspace registers.

* This research was supported in part by the National Science Foundation Grants MCS-7909259 and MCS-8201083.

[1] All logarithms are to the base 2.

0022-0000/87 $3.00

Copyright © 1987 by Academic Press, Inc.
All rights of reproduction in any form reserved.

In the *tape input model*, input values are stored on a read-only input tape and may be accessed only in a sequential scan of the entire tape. Output is to a write-only tape. Workspace registers may hold input values, and counts of size no greater than N. To be consistent with the results of [4], the space S_w will be counted as the number of workspace registers, assuming one input value may be stored in each such register at a time.

We first present a straightforward algorithm for sorting in the register input model. It achieves an upper bound trade-off of $T \cdot S_b = O(N^2 \log N)$ over the range $\Omega(\log N) = S_b = O(N)$. While the trade-off is similar to the time-capacity product $T \cdot S = O(N^2 \log N)$ achieved by the branching program in [3], it is superior in that the branching program cannot be implemented efficiently on a unit-cost RAM. In [2], $T \cdot S_b = O(N^2 \log N)$ is obtained for the special cases of $S_b = \Theta(N)$ or $S_b = \Theta(\log N)$, but for general S_b in the range $\Omega(\log N) = S_b = O(N)$, the best $T \cdot S_b$ product achieved is $O(N^2(\log N)^2)$.

We next consider time-space trade-offs for selection in a tape input model. Munro and Paterson [4] have presented an algorithm for selection, but have measured time in terms of the number of passes over the input tape. When time is measured in comparisons, their algorithm realizes a trade-off of $T \log S_w = O(N \log N)$ only over the range $\Omega((\log N)^2) = S_w = O(2^{\sqrt{\log N}})$. We modify and extend their approach to realize the same trade-off over the considerably broader range of $\Omega((\log N)^2) = S_w = O(2^{(\log N)/(\log^* N)})$. Due to arguments of [4], our result is optimal in this range.

Finally, we address the question of whether the time-space trade-off for sorting is less advantageous in the tape model than in the register model. Surprisingly, the answer appears to be no. We use some of our results on selection in the tape model to achieve a trade-off of $T \cdot S_w = O(N^2)$ for sorting over the range $\Omega((\log N)^2) = S_w = O(N/\log N))$. The trade-off achieved in [4] is $T \cdot S_w = O(N^2 \log(S_w/(\log N)))$, when time is measured in comparisons. Again, due to an argument in [4], our trade-off is optimal in the model.

2. Sorting in the Register Model

In this section, we present a sorting algorithm that realizes the time-space trade-off $T \cdot S_b = O(N^2 \log N)$ over the range $\Omega(\log N) = S_b = O(N)$, in the register model. Our algorithm is a combination of heap sort and an extraction sort. A constant number of workspace registers are required as program control registers, and the remainder, s registers, are used as a priority queue. The set of input registers are treated as s blocks of N/s registers each. The priority queue is initialized to contain the address of the smallest element from each block. Upon extraction of the smallest element from the priority queue, a replacement is found in the input register block from which the extracted element was drawn. Hence each block of input registers has a representative in the priority queue, until all elements of the block have been output.

LEMMA 1. *The time to sort N numbers using s workspace registers is $O(N \log s + N^2/s)$.*

Proof. The total time required to find the smallest elements from each block is $O(N)$. Forming the initial heap from these elements uses $O(s)$ time. Extracting an element from the heap, and inserting its replacement, takes $O(\log s)$ time, while finding a replacement for an element uses $O(N/s)$ time. Since $O(N)$ elements must be inserted and extracted from the heap, the total time is $O(N \log s + N^2/s)$. ∎

The fewest registers that can be used by the algorithm is four, using a heap of size one. Since in any model based on full-word comparisons $T = \Omega(N \log N)$, we consider space up to $S_b = O(N)$. Hence we are limited to $s = S_b/\log N = O(N/\log N)$ registers at most.

THEOREM 1. *A time-space product of $T \cdot S_b = O(N^2 \log N)$ may be realized for sorting in a register model, where S_b is the number of bits in workspace registers, in the range $4 \log N \leqslant S_b = O(N)$.*

Proof. This follows from Lemma 1, noting that $s \log s = O(N)$. ∎

We note that this algorithm cannot be implemented efficiently in a tape model, because replacement elements for the heap may be drawn out of the input in essentially random order. We shall give an algorithm for the tape model in Section 4, but with a more limited space range: $\Omega((\log N)^2) = S_w = O(N/(\log N))$.

3. TIME-SPACE TRADE-OFFS FOR SELECTION

In this section we present a selection algorithm that uses time $O(N \log^* S_w + (N \log N)/(\log S_w))$ in both models. Thus it realizes the time-space trade-off $T \log S_w = O(N \log N)$ over the range $\Omega((\log N)^2) = S_w = O(2^{(\log N)/(\log^* N)})$. Our algorithm is based on, and extends, results presented by Munro and Paterson [4]. Their algorithm for selection in the tape model uses $O(N \log S_w + (N \log N)/(\log S_w))$ time. Thus it realizes the same $T \log S_w$ trade-off over the more narrow range of $\Omega((\log N)^2) = S_w = O(2^{\sqrt{\log N}})$.

We describe their algorithm briefly, and then present our modifications. The algorithm uses a pair of elements, called *filters*, between which the element to be chosen must fall. On each sequential pass through the input, information is gathered that allows for the choice of more refined filters at the end of the pass. On the initial pass there are N elements between the filters, and by the final pass this number is reduced to a number of elements that can be accommodated in registers. The desired element is then selected directly, using the linear-time selection algorithm of [1].

On each pass, elements falling between the filters are used to construct a sample from which the filters are chosen. For some fixed s, an *s-sample at level i* is a sorted set of s elements chosen from a population of $s2^i$ elements. An s-sample at level 0

consists of a whole set of size $s2^0$ in sorted order. An s-sample at level $i+1$ is formed by splitting a population of size $s2^{i+1}$, taking two s-samples at level i from the first and second half of the population, "thinning" each by retaining every second element, and then merging the two subsamples to form one sorted sample. During any one pass over the tape, the s-sample of the current population is constructed in a bottom-up fashion. When two s-samples at level i for subpopulations of size $s2^i$ have been formed, they are immediately thinned and merged, and thus replaced by the resulting s-sample at level $i+1$. Thus at any point during the pass following the completion of any s-sample, there is at most one s-sample at each level, plus the s-sample just completed.

Let k be the integer such that the desired element is the kth largest in the current population at the beginning of a pass. After the pass, the new filters are chosen in the following way. Consider the jth largest element in an s-sample at level i. Let L_{ij} and M_{ij} be respectively the least and most number of elements from the corresponding population that may appear strictly above it in the total order. Let the size of the current population be $n = s2^r$. (This size can be determined when the current population is scanned to form the samples.) In the s-sample for the entire current population, the new filters will be the uth and vth elements of the s-sample, where u is the greatest integer such that $M_{ru} < k$ and v is the least integer such that $L_{rv} > k$. It is shown in [4] that the choice of u and v is such that $O((n/s)\log(n/s))$ elements will remain between the new filters. The fewest number of registers that can be used by the algorithm is $\Omega(s \log N) = \Omega((\log N))^2$. One factor of $\log N$ comes from the depth of recursion in the sampling procedure. The other comes from the requirement that $s \geqslant c \log(N/s)$ in order for the population to be reduced in size from one iteration to the next.

In the Munro and Paterson algorithm the sorting of s-samples at level 0 is most expensive during the first pass, when N/s blocks of s elements must be sorted. This yields a total time of $O((N/s)(s \log s)) = O(N \log s) = O(N \log S_w)$ for the first pass alone, which dominates the total time for sorting s-samples at level 0 on all subsequent passes. In addition, there are $O((\log N)/(\log s))$ passes in total, with $O(N)$ work performed per pass in scanning the elements and forming s-samples at all levels $i > 0$. This gives a total of $O(N(\log N)/(\log s))$ additional work over all passes. Hence $T \log S_w$ will be $O(N(\log S_w)^2)$, which will be $\omega(N \log N)$ whenever $\log S_w = \omega(\sqrt{\log N})$.

The bottleneck for the Munro and Paterson algorithm comes in the early passes, when the number of elements between the filters is $\omega(N/\log s)$. We thus replace the early passes in the Munro and Paterson algorithm with passes that use a somewhat different sampling procedure. Our samples will be good enough so that filtering $N/(\log^{(a)} s)$ elements down to no more than $N/(\log^{(a-1)} s)$ elements can be accomplished in a constant number of passes, each of cost $O(N)$. Thus N elements can be reduced to $N/\log s$ elements in $O(N \log^* s)$ steps.

In the early passes of our algorithm, we avoid completely sorting the samples at level 0, and thus speed up the time for selection when S_w is large. Instead of completely sorting these samples, we compute a limited amount of quantile information.

Let d be a factor of s, and s/d a power of 2. We define an (s, d)-*sample at level* i as a sorted set of $\min\{s, d2^i\}$ elements chosen from a population of $s2^i$ elements. An (s, d)-sample at level 0 is sequence of d elements, containing the (sj/d)th largest element, for $j = 1, 2, \ldots, d$. If $i+1 \leqslant \log(s/d)$, then an (s, d)-sample at level $i+1$ is formed by merging two (s, d)-samples at level i. If $i+1 > \log(s/d)$, then an (s, d)-sample is formed as in the Munro and Paterson algorithm by thinning two samples at level i, and then merging. We give a generalization of Lemma 2 in [4] below.

LEMMA 2. *For* $i \geqslant \log(s/d)$,

$$L_{ij} = j2^i - 1 \qquad \text{and} \qquad M_{ij} \leqslant (i - \log(s/d) + j + s/d - 2)\, 2^i.$$

Proof. The proof is by induction. The basis has $i = \log(s/d)$. The (s, d)-sample at level i is derived from a population of s/d groups of s elements each. Each element in the (s, d)-sample represents a subgroup of s/d elements. The jth largest element in the sample will have at least $j\, s/d - 1$ elements larger than it, and at most $j\, s/d - 1 + (s/d - 1)(s/d - 1)$ elements larger than it. The latter follows, since in addition to the elements counted by L_{ij} there can be at most $(s/d - 1)$ larger elements for each of the $(s/d - 1)$ subpopulations at level 0 other than the one containing the jth element in the sample at level i. The inductive portion of the proof follows that in [4]. ∎

We next generate a generalization of Lemma 1 in [4].

LEMMA 3. *If at most* n *elements lie between the filters at the beginning of a pass, then the number of elements between the new filters will be less than*

$$2n/s \log(n/s) + 2n/d.$$

Proof. Values of u and v were chosen such that $v = \lceil k/2^r \rceil$ and $u = \lceil k/2^r \rceil - r + \log(s/d) - s/d + 1$. The number of elements between the uth and vth elements of the sample is at most

$$\begin{aligned} M_{rv} - L_{ru} - 1 &= ((r - \log(s/d) + s/d - 2) + (v - u))\, 2^r \\ &= (2r - 2\log(s/d) + 2s/d - 3)\, 2^r < (2\log(n/s) + 2s/d)\, n/s. \quad \blacksquare \end{aligned}$$

Our modified selection algorithm is the following. Let $a = \log^* s - 2$. Set $d = \log^{(a)} s$. Perform passes until the number of elements is reduced to $N/(\log^{(a)} s)$. Then decrement a and repeat the above until $a = 0$. At this point the number of elements remaining in consideration has been reduced to no more than $N/(\log s)$. Complete the selection with the algorithm of [4], i.e., with $d = s$.

LEMMA 4. *The time to perform selection in a set of size* N *using* s *workspace registers is* $O(N \log^* s + N(\log N)/(\log s))$.

Proof. For $\log S_w = \Omega(\sqrt{\log N})$, the algorithm will require a constant number of passes for each value of d less than s. This follows since for any given value of a, with $d = \log^{(a)} s$, one pass reduces $N/(\log^{(a+1)} s)$ elements to fewer than $N/(\log^{(a+1)} s)((2/s)\log N + 2/(\log^{(a)} s))$ elements, by Lemma 3. The time for each pass with $d < s$ will be $O(N)$, since each of $O(N/(s \log^{(a)} s)$ groups of s elements will require time $O(s \log \log^{(a-1)} s) = O(s \log^{(a)} s)$ to find the sample elements. Since each value of $d < s$ requires a constant number of passes, there will be $O(\log^* s)$ passes with $d < s$. There will be $O((\log N)/(\log s))$ passes with $d = s$, each of which requires time $O(N)$. ∎

THEOREM 2. *The above selection algorithm realizes a time-space trade-off of* $T \log S_w = O(N \log N)$ *in the range* $\Omega((\log N)^2) = S_w = O(2^{(\log N)/(\log^* N)})$, *for both models of computation.*

From [4, Theorem 4] we have that any P-pass algorithm to determine the median of N elements requires storage of at least $\Omega(N^{1/P})$ words. The time required for P passes is $\Omega(PN)$. Combining these with Theorem 2 yields the following corollary.

COROLLARY 1. *The time-space trade-off for selection in the tape model is* $T \log S_w = \Theta(N \log N)$ *over the range* $\Omega((\log N)^2) = S_w = O(2^{(\log N)/(\log^* N)})$.

4. SORTING IN THE TAPE MODEL

In the last section, we presented an algorithm for selection that realizes the same good trade-off in both models. This result prompts the question of whether there is a similar trade-off for sorting that holds in both models. In Section 2, we presented a sorting algorithm that realizes $T \cdot S_w = O(N^2)$ for the random access model, but it appears that it cannot be adapted to the tape model. The best previous algorithm, by Munro and Paterson [4], realizes a trade-off of $T \cdot S_w = O(N^2 \log S_w)$. Surprisingly, the answer to the above question is yes, and we present the desired algorithm for the tape model in this section.

Our algorithm may be viewed as a major refinement of the procedure suggested in Munro and Paterson [4]. With z storage locations available, their algorithm outputs $z - 1$ elements on each pass over the input. On the ith pass their algorithm determines the $z - 1$ elements between the $(i-1)(z-1)$th and the $i(z-1)$th elements. Using a priority queue, the ith pass will make $O(N + (N - iz) \log z)$ comparisons, since every time an element is added to the current group of $z - 1$ elements, the largest must be found and removed. Hence the total number of comparisons will be $O((N^2/z)(\log z))$ and $T \cdot S_w = O(N^2(\log z))$.

To overcome the above problem, we start the ith output pass with filters, between which all elements to be output on that pass must fall. An obvious strategy for determining the filters is to find the median of the population recursively until

the resulting set is of size $z-1$. This strategy is employed by Tompa [7] to yield a branching program. We note, however, that this strategy still requires too much time. We thus settle for finding *near-medians* to divide the population recursively. The near-median is obtained by calling the sampling procedure from the previous section once with $d=4$, and then choosing the jth largest element in the sample, where $j=\lfloor(1-1/d)\,s/2\rfloor$. By Lemma 2, there will be at least $n/2-n/(2d)-1$ elements larger than this element, and at most $n/2+n/(2d)+\log(nd/(4s^2))$ elements larger than this element.

We choose our value of s to be $\Theta(S_w/(\log N))$, which corresponds to what we need in the sampling procedure. We then use $z=s\log N$ elements in each group to be sorted.

LEMMA 5. *In both models, the above sorting procedure uses time* $O(N\log s+N^2/(s\log N))$.

Proof. The total time to sort groups of no more than $s\log N$ elements is $O((N/(s\log N))\ (s\log N)\ \log(s\log N))=O(N\log s)$. Each call on the sampling procedure uses $O(N)$ time, and $O(N/(s\log N))$ such calls are made. The number of passes over the input is proportional to the number of calls on the sampling procedure. Hence total time to partition the N elements into groups of no more than s elements is $O(N^2/(s\log N))$. ∎

As in the previous section, our algorithm must use $O(s\log N)=O((\log N)^2)$ registers. As in Section 2, we conclude that S_w cannot exceed $O(N/(\log N))$, and s cannot exceed $O(N/(\log N)^2)$.

THEOREM 3. *The above sorting algorithm realizes a time-space product of* $T\cdot S_w=O(N^2)$ *on a tape model over the range* $\Omega((\log N)^2)=S_w=O(N/(\log N))$.

We note that the range of space over which the trade-off holds is not as broad as the range of our sorting algorithm for the register model in Section 2. As before we can make use of a lower bound result of Munro and Paterson, to characterize asymptotic behavior in a comparison-based model. From [4, Theorem 1], we have that the least storage required by any P-pass sorting algorithm is $\Omega(N/P)$ words. Also, any P-pass algorithm requires time $\Omega(PN)$. Combining these with Theorem 3 gives

COROLLARY 2. *The time-space trade-off for sorting in the tape model is* $T\cdot S_w=\Theta(N^2)$.

5. CONCLUSION

We have established time-space trade-off upper bounds for sorting and selection in two models of computation. It is interesting that, over broad ranges of space, the

trade-offs are no better for the random access model than for the more restrictive tape input model. The only difference identified occurs in the range of space used in the sorting algorithms, with the tape model requiring a larger minimum amount.

ACKNOWLEDGMENT

The author would like to thank the referee for several suggestions that improved the presentation.

REFERENCES

1. M. BLUM, R. W. FLOYD, V. PRATT, R. L. RIVEST, AND R. E. TARJAN, Time bounds for selection, *J. Comput. System Sci.* **7** (1973), 448–461.
2. A. BORODIN AND S. COOK, A time-space tradeoff for sorting on a general sequential model of computation, *SIAM J. Comput.* **11**, No. 2 (1982), 287–297.
3. A. BORODIN, M. J. FISCHER, D. G. KIRKPATRICK, N. A. LYNCH, AND M. TOMPA, A time-space tradeoff for sorting on non-oblivious machines, *J. Comput. System Sci.* **22**, No. 3 (1981), 351–364.
4. J. I. MUNRO AND M. S. PATERSON, Selection and sorting with limited storage, *Theoret. Comput. Sci.* **12** (1980), 315–323.
5. J. E. SAVAGE AND S. SWAMY, "Space-time Tradeoffs for Oblivious Sorting and Integer Multiplication," Technical Report CS-32, Brown University, Providence, R.I., 1978.
6. M. TOMPA, Time-space tradeoffs for computing functions, using connectivity properties of their circuits, *J. Comput. System Sci.* **20** (1980), 118–132.
7. M. TOMPA, "Time-space Tradeoffs for Straight-line and Branching Programs," Ph.D. thesis, Technical Report 122/78, University of Toronto, 1978.